

Metaheurísticas para el *Location Routing Problem*

ALNS · MA|PM

Sebastián Andre Lizárraga Calderón Johannes Albert Loayza Huamán

Proyecto Final de Carrera — 2026

Implementación en C++17 de dos metaheurísticas para optimización logística:

ALNS (Adaptive Large Neighborhood Search) + **MA|PM** (Memetic Algorithm with Population Management)
validadas sobre benchmarks Prodhon ($n = 10\text{--}200$ clientes)

El Problema LRP (Location Routing Problem)

Tres decisiones simultáneas:

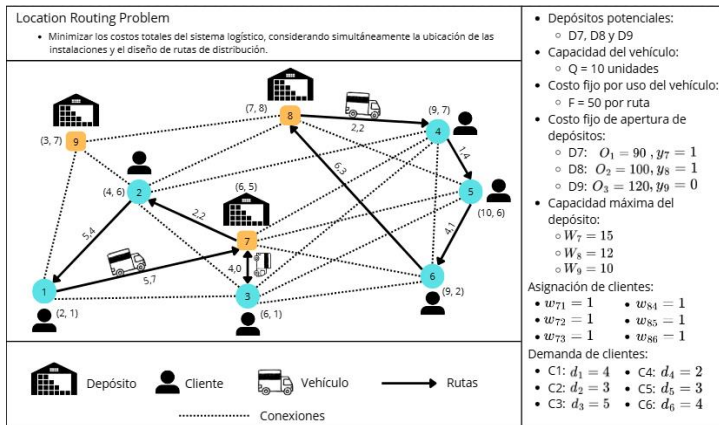
- 1 **Localización:** ¿qué depósitos abrir?
 $y_i \in \{0, 1\}$, costo O_i , cap. W_i
- 2 **Asignación:** ¿a qué depósito va cada cliente?
 $w_{ij} \in \{0, 1\}$
- 3 **Ruteo:** ¿en qué orden se visitan?
 $x_{ij} \in \{0, 1\}$, cap. Q , costo fijo F

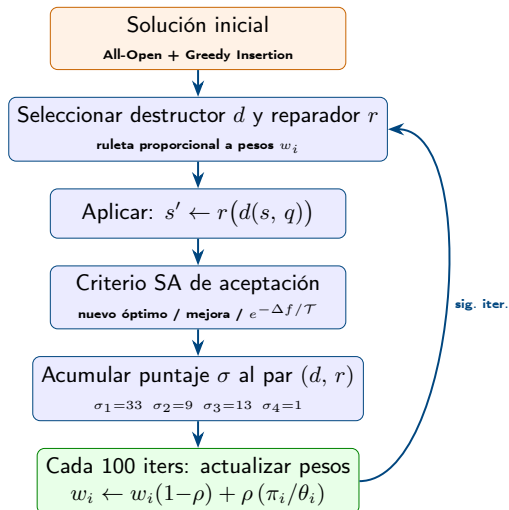
Función objetivo:

$$\begin{aligned} \min Z = & \sum_{i \in D} O_i y_i && \text{(apertura dep.)} \\ & + F \cdot |\mathcal{R}| && \text{(costo fijo)} \\ & + \sum_{(i,j)} c_{ij} x_{ij} && \text{(distancias)} \end{aligned}$$

Complejidad: NP-hard — combina FLP y VRP.

Las tres decisiones están **acopladas**: elegir otro depósito cambia toda la estructura de rutas.





Simulated Annealing (criterio de aceptación)

Solución peor s' aceptada con probabilidad $P = \exp(-(f(s') - f(s))/T)$.

Temperatura: $T \leftarrow \alpha T$ ($T_0 = 50\,000$, $\alpha = 0.998$).

Alta T : acepta casi todo (explora). Baja T : solo acepta mejoras (explota).

Ruleta Adaptativa (aprendizaje)

Cada par destructor+reparador acumula puntaje σ ($\sigma_1=33$, $\sigma_2=9$, $\sigma_3=13$, $\sigma_4=1$) según la calidad de la solución obtenida.

Cada 100 iters. los pesos se actualizan:

$$w_i \leftarrow w_i(1-\rho) + \rho(\pi_i/\theta_i), \quad \rho=0.1.$$

Operadores **efectivos** $\rightarrow$ mayor peso $\rightarrow$ mayor probabilidad de ser elegidos.

ALNS — Operadores de Destrucción y Reparación

Operadores de Destrucción (extraen q clientes a U)

- **Random removal:** extrae q clientes al azar. *Diversificación pura.*
- **Worst removal:** extrae los q de mayor costo marginal $\Delta_j = c_{aj} + c_{jb} - c_{ab}$. *Detecta clientes mal ubicados.*
- **Shaw removal:** extrae clientes similares (geográficos) a una semilla. *Exploita estructura espacial.*
- **Route removal:** elimina una ruta completa al azar. *Presiona reducción de flota.*
- **Depot closing:** cierra un depósito y libera todos sus clientes. *Única acción de localización.*

Operadores de Reparación (reinsertan clientes de U)

Greedy Insertion (miope)

Evalúa todas las posiciones en todas las rutas e inserta donde el costo $\delta = c_{aj} + c_{jb} - c_{ab}$ sea **mínimo**.

Regret-2 Insertion (previsor)

Calcula el *arrepentimiento* de cada cliente:
 $regret(j) = \delta^{(2)}(j) - \delta^{(1)}(j)$.

Inserta primero al de **mayor regret**: el que más perdería si espera. *Evita bloquear posiciones críticas.*

Nota: δ (inserción) y Δ (remoción) usan la misma fórmula $c_{aj} + c_{jb} - c_{ab}$ en sentido opuesto.

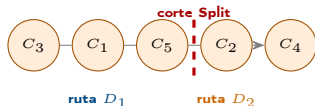
MA|PM — Representación y Decodificación (Split)

Codificación indirecta: el Cromosoma

En lugar de rutas explícitas, el MA|PM usa una representación de dos partes:

- DS** Vector de m enteros: $DS[i] > 0 \Rightarrow$ depósito i **abierto**.
- CS** Permutación de los n clientes (*Giant Tour*): un recorrido continuo sin indicar depósitos ni rutas.

Ejemplo con $n = 5$, $m = 2$, D_1 y D_2 abiertos:



El Giant Tour no dice nada sobre rutas. Eso lo decide el **Split**.

Algoritmo Split — decodificador óptimo

Construye un **DAG auxiliar** de $n+1$ nodos (posiciones de corte en el CS). Existe un arco $u \rightarrow v$ si los clientes $\pi_{u+1} \dots \pi_v$ forman una ruta factible ($\sum d \leq Q$), con peso igual al costo de esa ruta desde el depósito óptimo.

El **camino más corto** $0 \rightarrow n$ (Bellman-Ford) determina los **puntos de corte** (partición en rutas) y el **depósito** de cada segmento. Complejidad: $\mathcal{O}(n^2m)$.

¿Por qué es brillante el Split?

El cruce genético opera sobre **permutaciones de clientes** sin preocuparse por factibilidad. El Split convierte cualquier Giant Tour en rutas y asignaciones **óptimas**.

Cruce OX (Order Crossover)

1. Copiar segmento $[\pi_{k_1} \dots \pi_{k_2}]$ del Padre 1 al hijo.
2. Completar con clientes del Padre 2 en orden, omitiendo duplicados.

Preserva el orden relativo, que codifica la estructura espacial.

Búsqueda Local (fase de educación)

Aplicada a cada hijo tras Split (*first-improvement*):

- **Relocate:** mover un cliente a otra ruta o depósito.
- **Swap:** intercambiar dos clientes de rutas distintas.
- **2-opt:** invertir segmento intra-ruta (elimina cruces).

Cada individuo llega a la población ya “educado”.

PM — Control de Diversidad (BPD)

La **Broken Pairs Distance** cuenta pares adyacentes distintos entre dos soluciones:

$$BPD(A, B) = |\text{pares}(A) \Delta \text{pares}(B)|.$$

Un hijo entra si $BPD(\text{hijo}, P) \geq \Delta$. Demasiados rechazos: $\Delta \leftarrow \max(1, \Delta - 1)$.

Evita que la población degenera en clones.

Flujo del algoritmo

1. Inicializar P (Savings + aleatorio)
2. Loop: torneo $\rightarrow$ OX $\rightarrow$ Split $\rightarrow$ LS $\rightarrow$ BPD $\rightarrow$ insertar
3. Al estancarse: reinsertar élite y refrescar P
4. Parar si $NbAcc > MaxNbAcc$

Parámetros ($n=20, m=5$): $\mu=6$ indiv., $MaxNbAcc=250$,

$\Delta_{max}=12$.

Resultados Experimentales y Conclusiones

Validación exacta (instancias pequeñas)

Instancia	n	m	ALNS	MA PM
coord10-2-1	10	2	11 576	11 576
coord10-2-2	10	2	15 912	15 912
coord12-2-1	12	2	15 123	15 123

Gap = 0% en todos los casos respecto al óptimo exacto (Fuerza Bruta). Ambas implementaciones son correctas.

Benchmark Prodhon — 30 instancias con BKS

Escala	#inst.	Victorias		CPU ALNS
		ALNS	MA PM	
$n=20$	4	1	3	<0.1s
$n=50$	8	4	4	<0.3s
$n=100$	12	11	1	1–3s
$n=200$	6	6	0	6–13s
Total	30	22	8	—

MA|PM — instancias Prodhon $n=20$

Gana 3/4. Split + búsqueda local explotan el espacio en profundidad. Para $n=50$ es mixto (4–4): ambos compiten.

ALNS — instancias grandes ($n \geq 100$)

Gana 17/18. MA|PM tarda **100× más** con peores resultados. Única opción práctica a gran escala.

Conclusión

- Gap = 0% vs. óptimo exacto: **implementación verificada.**
- **MA|PM** gana en $n=20$; empate técnico en $n=50$.
- **ALNS** domina claramente en $n \geq 100$.
- Elección depende del tamaño y tiempo disponible.